
COMP206 Learning Journal

Hector Barquero

Jul 18, 2026

CONTENTS:

1	Assignment one journal	1
1.1	Program design	2
1.2	Overview and reflections	5
1.3	Readings	6
1.4	Practice problems	17
1.5	Sources and references	24
2	Assignment two journal	25
2.1	Program design	26
2.2	Overview and reflections	29
2.3	Readings	29
2.4	Practice problems	35
2.5	Sources and references	36
3	Assignment three journal	37
3.1	Program design	37
3.2	Overview and reflections	38
3.3	Readings	39
3.4	Practice problems	44
3.5	Sources and references	48
4	Assignment four journal	49

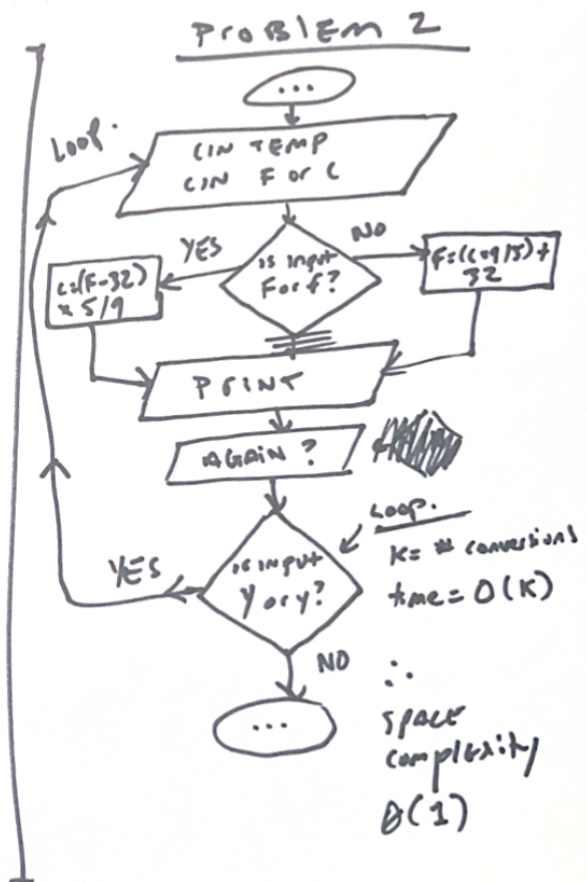
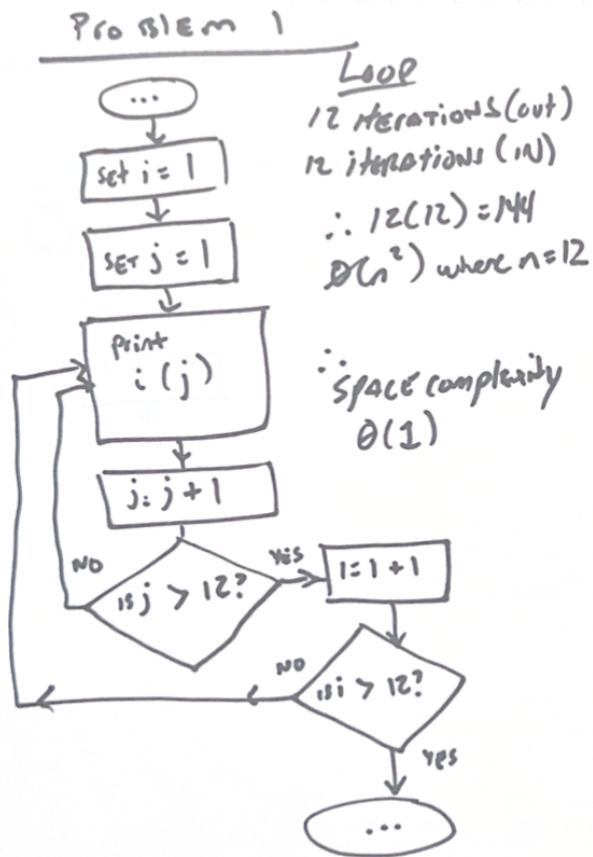
ASSIGNMENT ONE JOURNAL

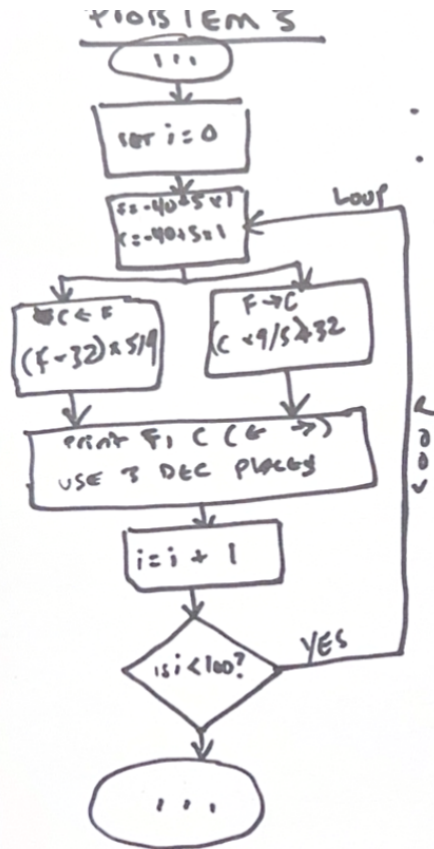
Time spent	Jan 2026 to Aug 2026
Student name	Hector Barquero

 Tip**How to use this journal**

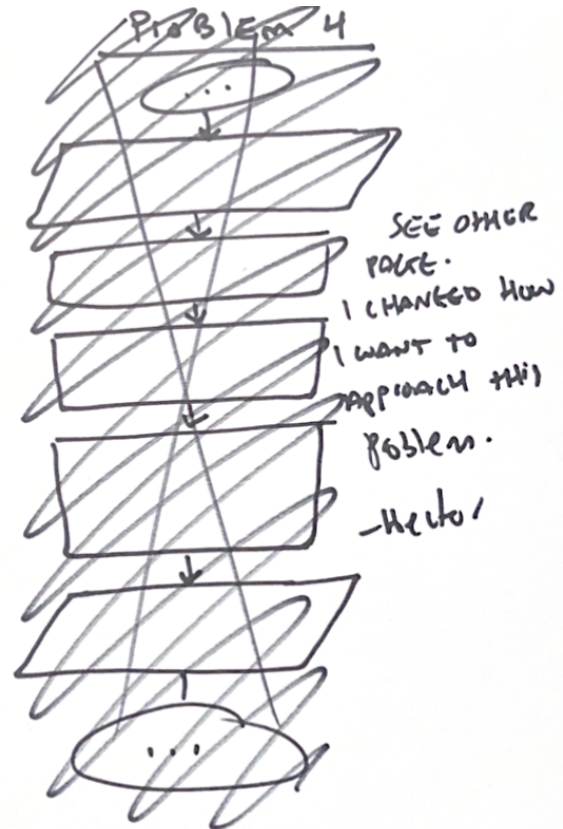
Each assignment has its own chapter. Use the right chapter to grade the assignment, which has program design notes, changelogs for versions as I made updates, reflections, and reading notes for the units of each assignment. There's also practice questions from the textbook which were recommended to be added to the learning journal for each assignment chapter. If a chapter is missing, it means that assignment isn't done yet because they're being completed sequentially— not all at the same time.

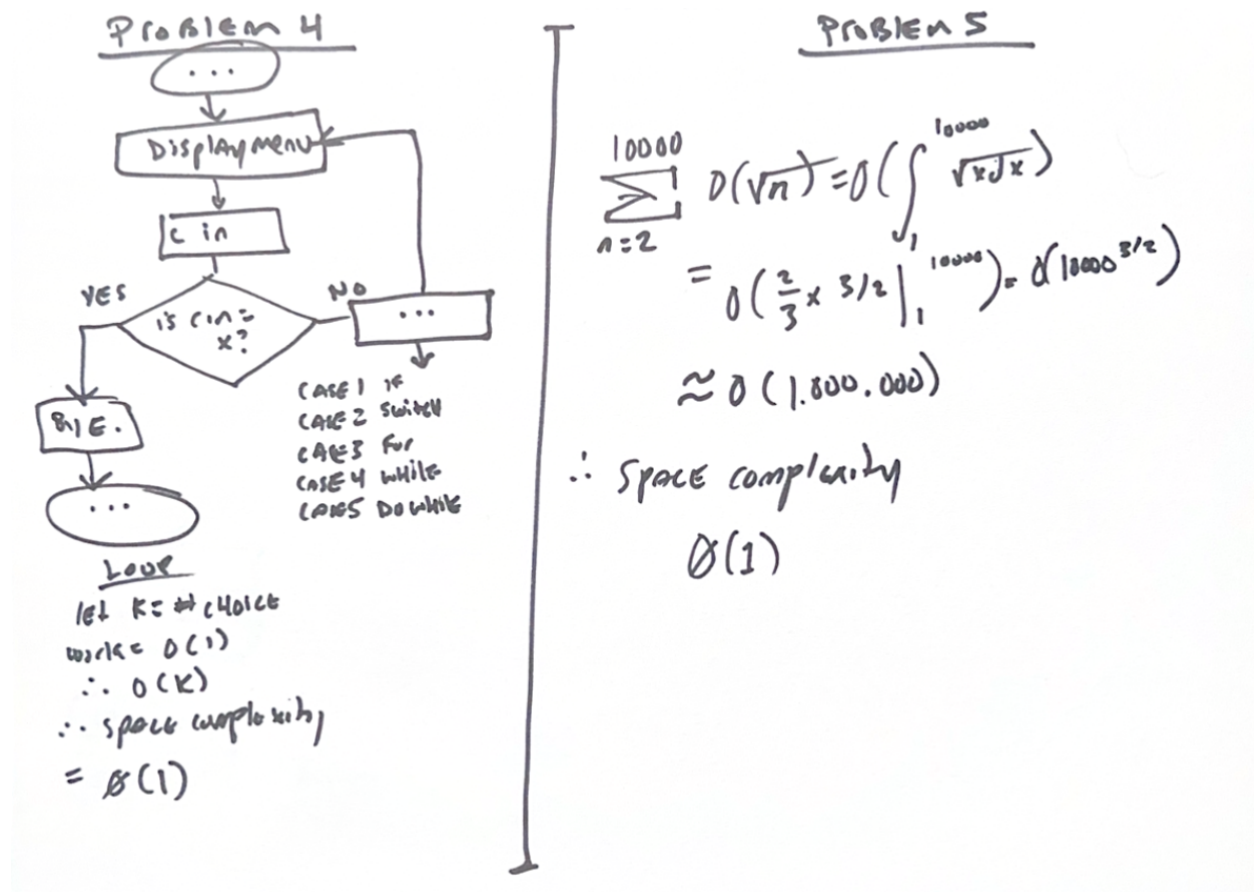
1.1 Program design





Loop
 - 100 iterations
 $O(1), O(n)$
 $\therefore$ SPACE complexity
 $O(1)$





1.1.1 Assignment 1, Problem 1

- Added nested for loops for the row and column values. (v1.0)
- Used setw to align three-character row labels and four-character products. (v1.1-v1.5)
- Compared the generated first and final rows with the supplied table. (v1.5)
- Corrected the first-column width so the vertical bars align with the sample.(v1.5-v1.7)

1.1.2 Assignment 1, Problem 2

- Added a repetition loop for multiple conversions. (v1.0)
- Implemented both temperature formulas using double values. (v1.0)
- Accepted uppercase and lowercase F, C, Y and N. (v1.0)
- Added input checks for non-numeric temperatures and unsupported units. (v1.2)
- Added extra support and redesigned program a bit to handle when user enters temp as string not int or double. (v1.2)
- Verified 32 F converts to 0 C, 100 C converts to 212 F and -40 converts to -40 in either direction. (v1.2)

1.1.3 Assignment 1, Problem 3

No changelog - was able to solve this in one attempt given similarity to previous problems.

- Added static functions for Fahrenheit-to-Celsius and Celsius-to-Fahrenheit. (v1.0)
- Generated exactly 100 rows beginning at -40 and increasing by 5. (v1.0)
- Used fixed, setprecision(3) and setw for table formatting. (v1.0)
- Verified the first row, the 100-degree row and the final 455-degree row. (v1.0)

1.1.4 Assignment 1, Problem 4

- Added a repeating menu implemented with a do-while loop. (v1.0)
- Added switch cases for if, switch, for, while and do-while. (v1.1)
- Added invalid-selection handling and support for x or X to exit. (v1.2)
- Verified each menu option and both exit characters. (v1.3)

1.1.5 Assignment 1, Problem 5

- Added nested for loops and the modulus operator for prime detection. (v1.0)
- Began candidate testing at 2 because 1 is not prime. (v1.1)
- Limited divisor testing to divisors whose square is no greater than the candidate. (v1.2)
- Formatted the output as ten prime numbers per row. (v1.2)
- Verified that 2 is the first prime, 9973 is the final prime and 1,229 primes are produced from 1 through 10,000. (v1.3)

1.2 Overview and reflections

1.2.1 Unit 1: Points to ponder

Hector Barquero posted Feb 27, 2026 8:12 AM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/337601/View?ou=18658>

I've programmed in C, C++ a lot and reading these feels like a nice refresher.

I've been programming in javascript a lot lately, so I almost forget definitions of things and some of the differences feel jarring. I caught myself forgetting that C++ doesn't do hoisting, and I forgot how strict it is with type declaration... Reading this content again has me remembering I have some javascript habits to break.

I also appreciate that both Think C++ and The Rooks Guide especially are really short and direct. I thought at first that they were quite dated, but I guess depending where you work... the C++ standard you might be writing could be dated too. (Where I am, I think it's an archaic standard, and the job previous to that we weren't using the latest either)

1.2.2 Unit 2: Points to ponder

Hector Barquero posted Feb 27, 2026 10:19 AM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/337650/View>

Completing unit 2 was interesting because it has differences to modern c++ that I needed to refresh on. I recall learning the same traditional academic declarations when first studying c++ at Fanshawe College in 2016:

```
//academic and traditional declaration the books seems to prefer: int foo = 0;
```

but when I started working in 2018+, I used brace form since it's modern best practice with c++:

//brace form which i think is c++ v11 and on, considered best practice to protect narrowing `int foo{0};`

This might be another thing for me to remember since I'm used to protecting type narrowing when doing narrowing conversion, but I scanned ahead for other units and see that the text prefers explicit casting conversion.

This is all really good for me to read again because in javascript, everything is dynamically typed where the type belongs to the value, not the variables

1.2.3 Unit 3: Points to ponder

Hector Barquero posted Feb 27, 2026 1:27 PM <https://learning.athabasca.ca/d2l/le/18658/discussions/threads/337734/View>

This unit was a good read. I spent a lot of time between multiple books this unit (including a javascript one I read yearly; eloquent javascript). I found there was a lot of reading that I thought I knew in the c++ books, but then it became blurry as I applied it in programming practice questions and tests- mainly because the casting, conversion, implicit coercion and explicit casting/conversion.

C++ prefers very deliberate programmer directed type conversion, but unlike javascript, the coercion happens less frequently.

javascript mutates the types more freely so coercion feels like something to always watch for, and programming explicit casting (conversion in js) is something normal.

I'm not sure how often I'll need to in this course, but I'll remember to use `static_cast<var>` or various named casts in my c++ programs, since the last time I touched it we used the more archaic `int foo = (int)bar; ...` a C style cast which doesn't seem to be used or popular in the books anymore.

1.3 Readings

1.3.1 Unit 1

ThinkCScpp 1.1: What is a programming language?

- A programming language provides formal rules for expressing instructions a computer can execute.
- High-level languages like C++ must be translated into machine language before they can run.

ThinkCScpp 1.2: What is a program?

- A program is a sequence of instructions that performs computation, input, output, testing, and repetition.

ThinkCScpp 1.3: What is debugging?

- Debugging is the process of finding and correcting errors in a program.
- Errors generally fall into compile-time, run-time, and logic categories.
- Debugging requires testing assumptions rather than randomly changing code.

ThinkCScpp 1.3.1: Compile-time errors

- Compile-time errors prevent the compiler from translating the source code.
- They commonly result from invalid syntax, missing punctuation, or incorrect declarations.

ThinkCScpp 1.3.2: Run-time errors

- Run-time errors occur after compilation while the program is executing.
- A program might crash, stop unexpectedly, or perform an invalid operation.
- The compiler can't always detect conditions that only appear during execution.
- Testing different inputs helps identify when the failure occurs.

ThinkCScpp 1.3.3: Logic errors and semantics

- Logic errors produce incorrect results even though the program compiles and runs.
- Semantic errors happen when valid code doesn't express the intended meaning.

ThinkCScpp 1.3.4: Experimental debugging

- Experimental debugging uses controlled tests to confirm or reject explanations for an error.
- Debug output can reveal variable values and show which statements execute.
- Changing several things at once makes it harder to identify the actual cause.

ProgFund 1: Introduction to Programming

- Programming converts a problem-solving process into instructions a computer can follow.
- Programs typically accept input, process data, and produce output.
- Good programming requires planning, implementation, testing, and maintenance.

ProgFund 1.1: Systems Development Life Cycle

- The SDLC organizes software work into planning, analysis, design, implementation, testing, and maintenance.
- Each phase reduces uncertainty before more development effort is committed.

ProgFund 1.3: Modularization and C++ Program Layout

- Modularization divides a large problem into smaller functions with specific responsibilities.
- A C++ program normally contains directives, declarations, functions, and a `main` function.
- Smaller modules are easier to test, reuse, and understand.
- Clear layout helps readers recognize the program's structure quickly.

ProgFund 2: Program Planning & Design

- Program planning defines the problem and solution before code is written.

ProgFund 2.1: Program Design

- Program design identifies inputs, processing steps, outputs, and required decisions.
- A planned solution is easier to implement and test than an improvised one.
- Complex problems should be divided into manageable modules.

ProgFund 2.2: Pseudocode

- Pseudocode describes program logic using structured language without strict C++ syntax.
- It helps refine algorithms before implementation details become a distraction.

ProgFund 2.3: Test Data

- Test data checks whether a program handles normal, boundary, and invalid inputs correctly.
- Expected results should be decided before the program is executed.
- One successful test can't prove that every possible input works.
- Carefully selected cases can expose incorrect conditions and assumptions.
- Tests should be repeated after changes to detect regressions.

ThinkCScpp 1.4: Formal and natural languages

- Natural languages tolerate ambiguity, while formal languages require precise syntax and structure.
- C++ can't infer intended meaning when its grammatical rules aren't followed.

ThinkCScpp 1.5: The first program

- A basic C++ program defines `main` as its starting point.
- `cout` writes output through the standard output stream.
- Statements generally end with semicolons.
- Compilation and execution are separate steps.

ProgFund 5: Integrated Development Environment

- An IDE combines tools for editing, compiling, running, and debugging programs.
- It simplifies development but doesn't replace understanding the build process.

ProgFund 5.1: Integrated Development Environment

- IDEs provide features such as syntax highlighting, project management, and error navigation.
- The compiler still determines whether C++ source code is valid.
- Debuggers let programmers inspect execution without adding permanent output statements.

ProgFund 5.2: Standard Input and Output

- `cin` reads values from standard input and `cout` writes values to standard output.
- Stream operators indicate whether data moves into or out of a stream.
- Input must match the destination variable's expected type.
- Output can combine text, values, and formatting within one statement.

ProgFund 5.3: Compiler Directives

- Preprocessor directives begin with `#` and are processed before compilation.
- `#include` makes declarations from headers available to the source file.

1.3.2 Unit 2

ThinkCScpp 2.1: More output

- Multiple values can be sent to `cout` using consecutive insertion operators.
- Escape sequences represent characters such as newlines and tabs.
- Output statements don't automatically add spacing between values.

ThinkCScpp 2.2: Values

- A value is a basic piece of data such as an integer, character, or string.
- Every value has a type that determines its meaning and permitted operations.

ThinkCScpp 2.3: Variables

- A variable is a named storage location associated with a specific type.
- Variables must be declared before they can be used.
- A variable's current value can change during execution.
- Using descriptive names makes expressions easier to understand.

ProgFund 3: Data & Operators

- Data types define how values are represented and which operations are valid.
- Operators combine or modify values to produce new results.

ProgFund 3.1: Data Types in C++

- C++ includes types for integers, floating-point numbers, characters, Boolean values, and text.
- Choosing an appropriate type affects memory use, precision, range, and available operations.
- C++ won't treat every type as interchangeable without a conversion.

ProgFund 3.2: Identifier Names

- Identifiers name variables, functions, types, and other program elements.
- An identifier can't begin with a digit or match a reserved keyword.
- Consistent naming conventions make related identifiers easier to recognize.

ProgFund 3.3: Constants and Variables

- Variables may change, while constants prevent reassignment after initialization.
- Constants communicate that a value isn't expected to change.
- `const` lets the compiler enforce that intention.
- Both constants and variables require suitable types.

ProgFund 3.4: Data Manipulation

- Expressions manipulate data through operators, variables, constants, and function calls.

ThinkCScpp 2.4: Assignment

- Assignment stores the right-hand value in the variable on the left.
- Assignment replaces the variable's previous value.
- The assignment operator doesn't mean mathematical equality.

ThinkCScpp 2.5: Outputting variables

- Variables can be inserted into an output stream alongside literal text.
- Output reflects the variable's value at the time the statement executes.

ThinkCScpp 2.6: Keywords

- Keywords have predefined meanings in C++ and can't be used as identifiers.
- Examples include `int`, `return`, `if`, and `while`.
- Editors often highlight keywords, but the compiler enforces their meaning.
- Keyword spelling and capitalization must be exact.

ProgFund 3.5: Assignment Operator

- The assignment operator evaluates the right-hand expression before storing its result.
- The left side must refer to a writable storage location.

ProgFund 3.6: Arithmetic Operators

- C++ supports addition, subtraction, multiplication, division, and remainder operations.
- The operand types affect how an arithmetic expression is evaluated.
- Division between integers discards the fractional part.
- Parentheses can make the intended grouping explicit.
- Invalid arithmetic, such as integer division by zero, can't produce a useful result.

ProgFund 3.7: Data Type Conversions

- Conversions change a value from one type to another.
- Implicit conversions happen automatically, while casts request a conversion explicitly.
- Narrowing conversions can lose range or precision.

ProgFund 4: Often Used Data Types

- Frequently used C++ types include integers, floating-point values, characters, strings, and Booleans.

ProgFund 4.1: Integer Data Type

- Integer types represent whole numbers without fractional components.
- Their supported range depends on the type and implementation.
- Signed types support negative values, while unsigned types don't.
- Overflow can produce incorrect or implementation-dependent behaviour.

ProgFund 4.2: Floating-Point Data Type

- Floating-point types represent values with fractional components and wide ranges.
- Many decimal fractions can't be represented exactly in binary.
- `double` generally offers more precision than `float`.

ProgFund 4.3: String Data Type

- `std::string` stores and manipulates sequences of characters.
- Strings support operations such as concatenation, comparison, indexing, and length checks.

RooksGuide 2: Variables

- Variables associate names with typed storage that a program can read and modify.
- C++ variables must be declared because their types are determined before execution.
- Initialization gives a variable its first value.
- Uninitialized local variables can contain indeterminate data.

RooksGuide 2.1: How do I decide which data type I need?

- Choose a type based on the required values, precision, operations, and range.
- The smallest possible type isn't always the clearest or safest choice.

RooksGuide 2.2: Identifiers

- Identifiers should describe their purpose without conflicting with C++ naming rules.
- Meaningful names reduce the need for explanatory comments.
- C++ identifiers are case-sensitive.

RooksGuide 2.3: Declaring a Variable

- A declaration introduces a variable's name and type to the compiler.

RooksGuide 2.4: Initializing Variables

- Initialization assigns a value when a variable is created.
- Brace initialization can reject some conversions that would lose information.
- Initializing variables early prevents accidental use of unknown values.

ThinkCScpp 2.7: Operators

- Operators perform computations using one or more operands.
- The same operator may behave differently with different types.
- Some operators modify values, while others only calculate results.
- Operator misuse can still compile when the expression is syntactically valid.

ThinkCScpp 2.8: Order of operations

- Operator precedence determines which parts of an expression are evaluated first.
- Parentheses should be used when the intended order isn't immediately clear.

ThinkCScpp 2.9: Operators for characters

- Character values can be compared and manipulated using their numeric encodings.
- Arithmetic on characters usually produces an integer result.
- Character ordering depends on the encoding used by the implementation.

ProgFund 4.4: Arithmetic Assignment Operators

- Operators such as `+=` and `*=` combine arithmetic with assignment.
- `x += 2` is generally equivalent to `x = x + 2`.
- Compound assignment avoids repeating the variable name.
- The usual type conversion rules still apply.

ProgFund 4.5: Lvalue and Rvalue

- An lvalue identifies an object or storage location that may appear on an assignment's left side.
- An rvalue is commonly a temporary value or expression result.
- Assignment requires a modifiable lvalue as its destination.

ProgFund 4.6: Integer Division and Modulus

- Integer division removes the fractional portion of the result.
- The modulus operator returns the remainder from integer division.
- Modulus is useful for checking divisibility and extracting digits.
- Negative operands can make remainder results less intuitive.

RooksGuide 2.5: Assignment Statements

- Assignment changes an existing variable's value after it has been declared.
- The source expression is evaluated before the destination is updated.

RooksGuide 3: Literals and Constants

- Literals write fixed values directly in source code, while named constants give those values meaning.
- Constants prevent values from being changed accidentally.
- Replacing unexplained literals with named constants improves maintainability.

RooksGuide 3.1: Literals

- A literal represents a value directly, such as `42`, `3.14`, `'A'`, or `"text"`.
- Literal syntax helps determine the value's type.

RooksGuide 3.2: Declared Constants

- A declared constant associates an immutable value with a descriptive identifier.
- Constants should usually be initialized when declared.
- `const` allows the compiler to reject later assignments.
- Named constants avoid repeating unexplained values.
- Changing one declaration can update every expression that uses the constant.

RooksGuide 4: Assignments

- Assignment stores an expression's result in an existing object.
- Chained assignments are possible but can reduce readability.
- Assignment isn't interchangeable with initialization in every context.

ThinkCScpp 2.10: Composition

- Composition combines variables, literals, operators, and function calls into larger expressions.
- Complex expressions should remain readable and avoid repeating unnecessary calculations.

1.3.3 Unit 3**ThinkCScpp 3.1: Floating-point**

- Floating-point values approximate real numbers using limited binary precision.
- `double` is commonly preferred when fractional calculations need reasonable precision.
- Equality comparisons can fail when rounding produces slightly different results.

ThinkCScpp 3.2: Converting from double to int

- Converting a `double` to `int` discards the fractional part rather than rounding it.
- Values outside the integer type's range can't be represented safely.
- An explicit cast makes the conversion visible to readers.
- Rounding functions should be used when truncation isn't intended.

ThinkCScpp 3.3: Math functions

- The `<cmath>` header provides functions for roots, powers, rounding, and trigonometry.
- Math functions accept arguments and return calculated results.

ThinkCScpp 3.4: Composition

- Function calls can be nested inside expressions and passed directly as arguments.
- Composition can remove temporary variables but shouldn't make code difficult to read.
- Each inner expression is evaluated before its result is used by the outer expression.

ThinkCScpp 3.5: Adding new functions

- Functions package a named sequence of statements into a reusable operation.
- A function definition specifies its return type, name, parameters, and body.
- Decomposing code into functions reduces duplication.
- Function names should describe the operation being performed.
- `main` can call functions that were declared before the call.

ThinkCScpp 3.6: Definitions and uses

- A function must be declared or defined before the compiler encounters a call to it.
- A declaration describes the function's interface without providing its implementation.

ThinkCScpp 3.7: Programs with multiple functions

- Multiple functions divide a program into smaller units with separate responsibilities.
- Execution begins in `main` and moves into other functions when they're called.
- Functions can call other functions when their declarations are visible.

ThinkCScpp 3.8: Parameters and arguments

- Parameters are variables declared by a function, while arguments are values supplied by the caller.
- Arguments are matched to parameters by position and compatible type.
- Passing by value gives the function its own copy of the argument.

ThinkCScpp 3.9: Parameters and variables are local

- Parameters and variables declared inside a function are local to that function.
- Separate functions can use the same local name without sharing storage.
- Local variables stop existing when their scope ends.
- A function can't directly access another function's local variables.

ThinkCScpp 3.10: Functions with multiple parameters

- Functions can accept multiple parameters separated by commas.
- Each parameter requires its own declared type.
- Arguments must be supplied in the expected order.

ThinkCScpp 3.11: Functions with results

- A non-void function returns a value using a `return` statement.
- The returned value's type must be compatible with the declared return type.
- Returned results can be stored, printed, or used inside larger expressions.
- Every reachable path should return a value when the function isn't `void`.
- Returning a result often makes a function easier to reuse than printing internally.

ProgFund 6: Program Control Functions

- Program control functions organize execution by separating tasks into callable modules.
- Functions can receive data, perform processing, and return results.

ProgFund 6.1: Pseudocode Examples for Functions

- Function pseudocode describes parameters, processing steps, and returned values without C++ syntax.
- It helps define responsibilities before implementation.
- Calls in the main algorithm show how individual modules interact.

ProgFund 6.2: Hierarchy or Structure Chart

- A structure chart shows which functions call other functions.
- It represents program organization without describing every statement.
- Higher-level functions coordinate work performed by lower-level modules.
- The chart can expose modules that are too large or tightly connected.

ProgFund 6.3: Program Control Functions

- Control functions coordinate other functions rather than performing every task directly.

ProgFund 6.4: Void Data Type

- A void function performs an action without returning a value.
- void can also indicate that a function doesn't accept parameters in some declarations.
- A void function may use return without an expression to exit early.

ProgFund 6.5: Documentation and Making Source Code Readable

- Readable code uses meaningful names, consistent formatting, and focused functions.
- Comments should explain intent or unusual decisions rather than repeat the code.
- Excessive comments can become inaccurate when the implementation changes.
- Documentation should describe interfaces, assumptions, and important constraints.
- Consistent style makes errors and structural problems easier to notice.

RooksGuide 5: Output

- Output streams send formatted values to destinations such as the console.
- `std::cout` uses `<<` to insert data into the standard output stream.

RooksGuide 6: Input

- `std::cin` uses `>>` to extract typed values from standard input.
- Failed input leaves the stream in an error state until it's handled.
- Whitespace-delimited extraction won't read an entire line containing spaces.
- `std::getline` is better suited to full-line text input.

RooksGuide 7: Arithmetic

- Arithmetic expressions follow C++ precedence and type-conversion rules.
- Integer and floating-point operands can produce different results.
- Parentheses clarify calculations and reduce reliance on remembered precedence.

RooksGuide 8: Comments

- Comments document intent and are ignored during compilation.
- Line comments begin with `//`, while block comments use `/*` and `*/`.
- Good names and structure should carry most of the explanation.
- Comments shouldn't preserve obsolete code that version control already stores.
- Misleading comments are worse than missing comments.

RooksGuide 9: Data Types and Conversion

- Data types define representation, range, precision, and supported operations.
- Conversions allow expressions to combine different types.
- Some conversions are safe, while others can lose information.

RooksGuide 9.1: Floating-point types

- `float`, `double`, and `long double` offer different minimum precision and range.
- Floating-point arithmetic is approximate and subject to rounding.
- `double` is generally the default type for decimal literals.
- More precision doesn't make every decimal value exact.

RooksGuide 9.2: Other types introduced by C++11

- C++11 added features such as fixed-width integers, `nullptr`, and improved type inference.
- Fixed-width integer types are useful when an exact bit width is required.

RooksGuide 9.3: Conversion Between Types

- Implicit conversions occur when C++ automatically changes a value's type.
- Promotions usually preserve information, while narrowing conversions might not.
- Mixed-type expressions convert operands to a common type.
- Conversion behaviour should be considered before assigning the result.

RooksGuide 9.4: Coercion & Casting

- Coercion is an automatic conversion performed by the language.
- Casting explicitly requests that a value be treated as another type.
- `static_cast` clearly expresses many ordinary compile-time conversions.

RooksGuide 9.5: Automatic Types in C++11

- auto asks the compiler to infer a variable's type from its initializer.
- The inferred type remains static and can't later change like a JavaScript variable.
- auto can reduce repetition when the initializer already makes the type clear.
- It shouldn't hide information that readers need to understand the code.
- An auto variable normally requires an initializer.

1.4 Practice problems

Tip

Below are only five of my favourite problems I solved for each unit. A comprehensive problemset is solved over the course of COMP206 and you can view my answers at :doc: *Practice Problems* <PracticeProblems.rst>

1.4.1 Unit 1 practice problems

Fizz Buzz (Easy)

Return numbers as strings, replacing multiples of 3 and 5 with the required words.

```
class Solution {
public:
    vector<string> fizzBuzz(int n) {
        vector<string> answer;

        for (int i = 1; i <= n; ++i) {
            if (i % 15 == 0) {
                answer.push_back("FizzBuzz");
            } else if (i % 3 == 0) {
                answer.push_back("Fizz");
            } else if (i % 5 == 0) {
                answer.push_back("Buzz");
            } else {
                answer.push_back(to_string(i));
            }
        }

        return answer;
    }
};
```

Number of Steps to Reduce a Number to Zero (Easy)

Count how many divisions and subtractions are required to reduce a number to zero.

```
class Solution {
public:
    int numberOfSteps(int num) {
        int steps = 0;
```

(continues on next page)

(continued from previous page)

```
while (num > 0) {
    if (num % 2 == 0) {
        num /= 2;
    } else {
        --num;
    }

    ++steps;
}

return steps;
}
};
```

Richest Customer Wealth (Easy)

Find the greatest sum among the rows of a two-dimensional integer array.

```
class Solution {
public:
    int maximumWealth(vector<vector<int>>& accounts) {
        int maximum = 0;

        for (const vector<int>& customer : accounts) {
            int total = 0;

            for (int account : customer) {
                total += account;
            }

            maximum = max(maximum, total);
        }

        return maximum;
    }
};
```

Valid Palindrome (Easy)

Determine whether normalized text reads the same forward and backward.

```
class Solution {
public:
    bool isPalindrome(string s) {
        int left = 0;
        int right = static_cast<int>(s.size()) - 1;

        while (left < right) {
            while (left < right && !isalnum(s[left])) {
                ++left;
            }
        }
    }
};
```

(continues on next page)

(continued from previous page)

```

        while (left < right && !isalnum(s[right])) {
            --right;
        }

        if (tolower(s[left]) != tolower(s[right])) {
            return false;
        }

        ++left;
        --right;
    }

    return true;
}
};

```

Two Sum (Easy)

Return the indexes of two array values whose sum equals the target.

```

class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> indexes;

        for (int i = 0; i < nums.size(); ++i) {
            int required = target - nums[i];

            if (indexes.count(required)) {
                return {indexes[required], i};
            }

            indexes[nums[i]] = i;
        }

        return {};
    }
};

```

1.4.2 Unit 2 practice problems

Final Value of Variable After Performing Operations (Easy)

Apply a sequence of increment and decrement operations to an integer variable.

```

class Solution {
public:
    int finalValueAfterOperations(vector<string>& operations) {
        int value = 0;

        for (const string& operation : operations) {

```

(continues on next page)

(continued from previous page)

```
        if (operation[1] == '+') {
            ++value;
        } else {
            --value;
        }
    }

    return value;
}
};
```

Subtract the Product and Sum of Digits of an Integer (Easy)

Return the product of an integer's digits minus their sum.

```
class Solution {
public:
    int subtractProductAndSum(int n) {
        int product = 1;
        int sum = 0;

        while (n > 0) {
            int digit = n % 10;
            product *= digit;
            sum += digit;
            n /= 10;
        }

        return product - sum;
    }
};
```

Count the Digits That Divide a Number (Easy)

Count how many digits divide the original integer without a remainder.

```
class Solution {
public:
    int countDigits(int num) {
        int count = 0;

        for (int remaining = num; remaining > 0; remaining /= 10) {
            int digit = remaining % 10;

            if (digit != 0 && num % digit == 0) {
                ++count;
            }
        }

        return count;
    }
};
```


Plus One (Easy)

Increment a large integer represented as an array of decimal digits.

```
class Solution {
public:
    vector<int> plusOne(vector<int>& digits) {
        for (int i = static_cast<int>(digits.size()) - 1; i >= 0; --i) {
            if (digits[i] < 9) {
                ++digits[i];
                return digits;
            }

            digits[i] = 0;
        }

        digits.insert(digits.begin(), 1);
        return digits;
    }
};
```

Valid Anagram (Easy)

Determine whether two strings contain identical character counts.

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) {
            return false;
        }

        array<int, 26> counts{};

        for (char character : s) {
            ++counts[character - 'a'];
        }

        for (char character : t) {
            --counts[character - 'a'];
        }

        for (int count : counts) {
            if (count != 0) {
                return false;
            }
        }

        return true;
    }
};
```

1.4.3 Unit 3 practice problems

Fibonacci Number (Easy)

Return the requested Fibonacci number using a function with a result.

```
class Solution {
public:
    int fib(int n) {
        if (n < 2) {
            return n;
        }

        int previous = 0;
        int current = 1;

        for (int i = 2; i <= n; ++i) {
            int next = previous + current;
            previous = current;
            current = next;
        }

        return current;
    }
};
```

Sqrt(x) (Easy)

Return the truncated integer square root of a non-negative integer.

```
class Solution {
public:
    int mySqrt(int x) {
        int left = 0;
        int right = x;
        int result = 0;

        while (left <= right) {
            int middle = left + (right - left) / 2;
            long long square = static_cast<long long>(middle) * middle;

            if (square <= x) {
                result = middle;
                left = middle + 1;
            } else {
                right = middle - 1;
            }
        }

        return result;
    }
};
```

Power of Two (Easy)

Determine whether an integer is an exact power of two.

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        if (n <= 0) {
            return false;
        }

        while (n % 2 == 0) {
            n /= 2;
        }

        return n == 1;
    }
};
```

Reverse Integer (Medium)

Reverse an integer's digits while returning zero when the result would overflow.

```
class Solution {
public:
    int reverse(int x) {
        int result = 0;

        while (x != 0) {
            int digit = x % 10;
            x /= 10;

            if (result > INT_MAX / 10 ||
                (result == INT_MAX / 10 && digit > 7)) {
                return 0;
            }

            if (result < INT_MIN / 10 ||
                (result == INT_MIN / 10 && digit < -8)) {
                return 0;
            }

            result = result * 10 + digit;
        }

        return result;
    }
};
```

Pow(x, n) (Medium)

Calculate a floating-point base raised to an integer exponent.

```
class Solution {
public:
    double myPow(double x, int n) {
        long long exponent = n;

        if (exponent < 0) {
            x = 1.0 / x;
            exponent = -exponent;
        }

        double result = 1.0;

        while (exponent > 0) {
            if (exponent % 2 == 1) {
                result *= x;
            }

            x *= x;
            exponent /= 2;
        }

        return result;
    }
};
```

1.5 Sources and references

1. cppreference.com. (n.d.). C++ reference. Retrieved Jan 2026 - Aug 2026, from <https://cppreference.com/cpp>
2. Braunschweig, D., & Busbee, K. L. (2018). Programming fundamentals: A modular structured approach (2nd ed.). Rebus Community. <https://press.rebus.community/programmingfundamentals/>
3. Hansen, J. A. (2013). The Rook's guide to C++. Rook's Guide Press. <https://rooksguide.org/>
4. Downey, A. B. (1999). Think C++ (Version 1.1.0). Green Tea Press. <https://www.greenteapress.com/thinkcpp/>
5. leetcode.com. (n.d.). *LeetCode*. Retrieved Jan 2025 - Aug 2026, from <https://leetcode.com/>

ASSIGNMENT TWO JOURNAL

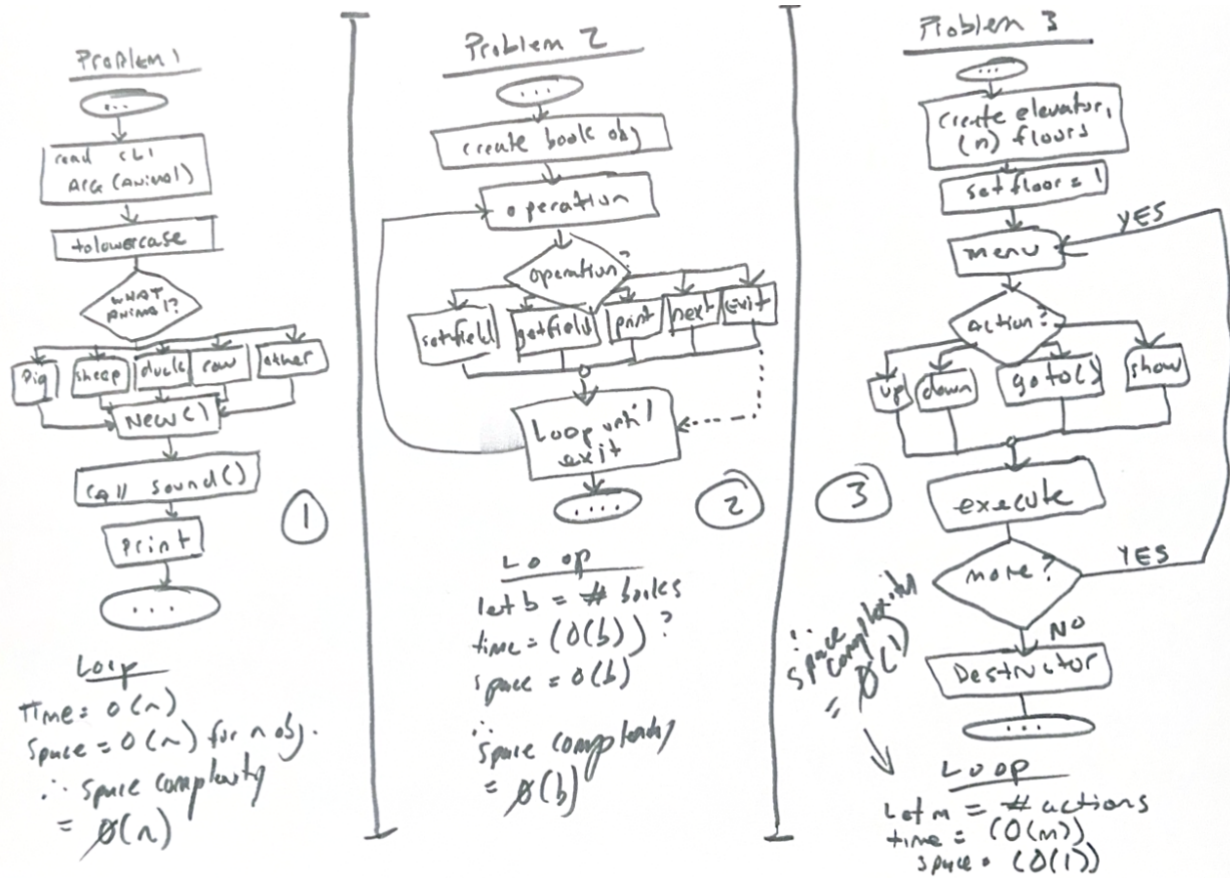
Time spent	Jan 2026 to Aug 2026
Student name	Hector Barquero

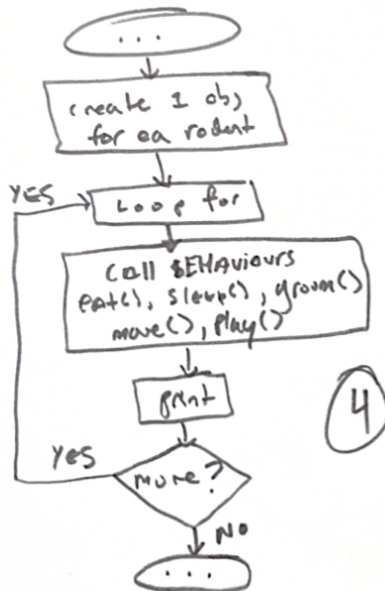
Tip

How to use this journal

Each assignment has its own chapter. Use the right chapter to grade the assignment, which has program design notes, changelogs for versions as I made updates, reflections, and reading notes for the units of each assignment. There's also practice questions from the textbook which were recommended to be added to the learning journal for each assignment chapter. If a chapter is missing, it means that assignment isn't done yet because they're being completed sequentially— not all at the same time.

2.1 Program design



Problem 4

Loop
 Time $O(r.b) = O(1)$ const.
 Space $O(1) = O(1)$ const.
 $\therefore$ space complexity = $O(1)$

Problem 5

USE HERON'S FORMULA TO SOLVE?
 Circle: $Area = \pi r^2$, $circumf = 2\pi r$
 Rect: $Area = w \times h$, $circ = 2(w+h)$
 is sq $\Rightarrow w = 4$ ($\Leftarrow \Rightarrow$)

HERON

$$a = |p_1 - p_2|, b = |p_2 - p_3|, c = |p_3 - p_1|$$

$$s = \frac{(a + b + c)}{2}$$

$$Area = \sqrt{s(s-a)(s-b)(s-c)}$$

$$circumf = a + b + c$$

$\therefore \min X = \min(\text{all } x), \max X = \max(\text{all } x)$
 SAME FOR min Y.

Loop

let $n = \#$ SHAPES created.
 $\Leftarrow$ EACH = $O(1)$, time = $O(n)$
 Space...

$\therefore$ space complexity ~
 $= O(n)$

2.1.1 Assignment 2, Problem 1

- Added an Animal base class with a default constructor and generic sound() method. (v1.0)
- Added Pig, Sheep, Duck and Cow child classes with animal-specific constructors and overridden sound() methods. (v1.0-v1.2)
- Created several instances of each animal and verified oink, baah, quack and moo output. (v1.2)
- Made sound() virtual so child sounds can be called through an Animal pointer. (v1.3)
- Added an AnimalTest class that creates the correct child object from command-line input. (v1.3-v1.5)
- Added support for mixed-case animal names and error handling for unknown animals or extra arguments. (v1.5-v1.7)
- Used `unique_ptr<Animal>` so the selected child object is cleaned up automatically. (v1.7)

2.1.2 Assignment 2, Problem 2

- Added a Book class with private title, ISBN, author, edition, publisher and publication-year attributes. (v1.0)
- Added public getter and setter methods for each private attribute. (v1.0-v1.2)
- Added a default constructor and a parameterized constructor for initializing Book objects. (v1.2-v1.3)
- Added display() to print all stored book information in a consistent format. (v1.3)
- Created three Book objects using both constructors and setter methods. (v1.3-v1.5)

- Verified that each object keeps its own values and that every getter returns the value assigned by its setter. (v1.5-v1.6)

2.1.3 Assignment 2, Problem 3

- Added an Elevator class that stores the total number of floors and the current floor. (v1.0)
- Added a default constructor for a five-storey building and a parameterized constructor for custom building sizes. (v1.0-v1.2)
- Added moveTo() to move between floors and reject destinations outside the building range. (v1.2-v1.4)
- Added handling for requests to move to the elevator's current floor. (v1.4)
- Added a destructor that returns the elevator to floor one and prints the required cleanup message. (v1.4-v1.5)
- Tested default construction, upward movement, downward movement, invalid movement and repeated movement. (v1.5-v1.7)
- Used unique_ptr::reset() to demonstrate when a dynamically created Elevator object is destroyed in C++. (v1.7)

2.1.4 Assignment 2, Problem 4

- Added a Rodent base class with common eat(), sleep(), groom() and move() behaviours. (v1.0)
- Added Mouse, Gerbil, Hamster and GuineaPig child classes. (v1.0-v1.2)
- Overrode only behaviours that differ for each rodent, while leaving common behaviours inherited from Rodent. (v1.2-v1.4)
- Added type() and demonstrateBehaviours() methods to identify each rodent and run all of its behaviours. (v1.4-v1.5)
- Stored the child objects in a vector of unique_ptr<Rodent> objects. (v1.5)
- Called each behaviour through Rodent pointers to verify runtime polymorphism. (v1.5-v1.7)
- Verified that inherited methods still run when a child class does not override a behaviour. (v1.7)

2.1.5 Assignment 2, Problem 5

- Added a Point class with separate x and y values and show(), add() and subtract() methods. (v1.0)
- Added a Shape base class with virtual area(), circumference(), boundingBox() and display() methods. (v1.0-v1.2)
- Added Circle, Rectangle and Triangle child classes with default and parameterized constructors. (v1.2-v1.4)
- Implemented area and circumference calculations for each valid shape. (v1.4-v1.5)
- Added axis-aligned bounding-box calculations using minimum and maximum x and y coordinates. (v1.5)
- Added constructor validation for positive circle radii, non-collinear triangles and four-sided shapes that satisfy rectangle rules. (v1.5-v1.7)
- Added floating-point tolerance checks so calculated lengths and angles are not compared using exact equality. (v1.7)
- Added square detection when all four rectangle sides have equal length. (v1.7-v1.8)
- Tested valid circles, rectangles and triangles, along with an invalid circle, a non-rectangle quadrilateral, a square and a collinear triangle. (v1.8-v2.0)

2.2 Overview and reflections

2.2.1 Unit 4: Points to ponder

Hector Barquero posted Mar 4, 2026 3:41 PM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/341404/View>

This unit goes over some fundamentals that were covered in depth within COMP200. If you read the COMP200 text then you'd be pretty prepared to go through this unit with little difficulty.

Control flow and conditions are very familiar to whichever language you may already know, since the history of structured programming and C-based adoption

2.2.2 Unit 5: Points to ponder

Hector Barquero posted Mar 4, 2026 4:18 PM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/341427/View>

This unit went further into recursion. One of the most interesting things this unit was remembering overloading.

Javascript doesn't do this conventionally. You can write a design pattern to simulate overloading, but ultimately it's run-time evaluation whereas C++ is compile time. You can overload in C++ which is really handy, and something I almost forgot.

This will let me enjoy similar js-like flexibility when writing functions, since multiple functions can share the same name but with different parameters.

2.2.3 Unit 6: Points to ponder

Hector Barquero posted Mar 21, 2026 7:13 AM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/350429/View>

This unit was a lot of reading, especially the additional and further reading. One of the most interesting parts I learned from reading it all was that loop choice encodes intent, and not capability. I've been selecting the wrong control flow all this time and now see that even though it expresses the same logic, maintenance and readability suffers.

I should be using bounded iteration (for) where it makes sense, not by default. Condition driven control (while), and once executed (do while) make a lot more sense for state transitions when I go back to read my code, but I must have picked up a habit of using bounded iteration primarily.

It also helps because the cognitive load of reading it is a bit worse when choosing the wrong control flow design. You can get earlier returns with the right one.

2.3 Readings

2.3.1 Unit 4

ThinkCScpp 4.1: The modulus operator

- The modulus operator % returns the remainder from integer division.
- It is useful for divisibility tests, digit extraction, and repeating numeric patterns.

ThinkCScpp 4.2: Conditional execution

- An if statement runs a block only when its condition evaluates to true.
- Conditions usually use comparison or logical operators.
- Conditional execution lets a program choose whether an action should occur.

ThinkCScpp 4.3: Alternative execution

- An `if` and `else` statement selects exactly one of two possible paths.
- The `else` block runs only when the original condition is false.

ThinkCScpp 4.4: Chained conditionals

- An `if` followed by `else if` branches can test several mutually exclusive cases.
- Conditions are checked from top to bottom until one succeeds.
- The final `else` can handle all remaining cases.
- Ordering matters when conditions overlap.

ThinkCScpp 4.5: Nested conditionals

- A nested conditional places one decision inside another.
- Nesting is useful when a second test only matters after the first succeeds.
- Deep nesting can usually be simplified with combined conditions or early returns.

ThinkCScpp 4.6: The return statement

- A `return` statement immediately ends the current function.
- A value-returning function uses `return` to send a result back to its caller.

ThinkCScpp 4.7: Recursion

- Recursion occurs when a function calls itself to solve a smaller version of a problem.
- Every recursive function needs a base case that stops further calls.
- The recursive case must move toward the base case.
- Recursive solutions often mirror mathematical definitions or nested structures.

ThinkCScpp 4.8: Infinite recursion

- Infinite recursion happens when recursive calls never reach a stopping condition.
- It eventually exhausts the call stack and causes the program to fail.
- Common causes include missing base cases and arguments that don't move toward termination.

ThinkCScpp 4.9: Stack diagrams for recursive functions

- A stack diagram shows each active function call and its local variables.
- Every recursive call receives its own separate parameters and local storage.
- Calls are removed from the stack in reverse order when they return.
- Stack diagrams help trace how recursive results are assembled.

RooksGuide 10: Conditionals

- Conditionals control which statements execute based on Boolean expressions.
- They allow programs to react differently to different data.
- Clear conditionals avoid duplicated logic and unnecessary nesting.

RooksGuide 10.1: if, else, and else if

- `if` handles the first condition, `else if` checks alternatives, and `else` handles the remainder.
- Only the first matching branch in a chain executes.
- Braces make each branch's scope explicit.
- Conditions should be ordered from most specific to most general.

RooksGuide 10.1.1: A small digression on expressions

- An expression combines values, variables, operators, and function calls to produce a result.
- Conditional expressions are converted to Boolean values.
- Nonzero numeric values act as true, while zero acts as false.

RooksGuide 10.1.2: Using else

- `else` provides a fallback path when its matching `if` condition is false.
- An `else` belongs to the nearest unmatched `if`.
- Braces prevent ambiguity in nested conditions.
- A final `else` can document that all possible cases were considered.

RooksGuide 10.2: switch statements

- A `switch` selects a branch by comparing one expression against constant case values.
- `break` usually prevents execution from falling through into the next case.
- `default` handles values that don't match any listed case.
- `switch` is useful for discrete choices but can't directly express arbitrary ranges.
- Duplicate case values aren't allowed.

2.3.2 Unit 5**ThinkCScpp 5.1: Return values**

- A function's return value is the result supplied to its caller.
- The returned expression must be compatible with the declared return type.
- Return values can be stored, printed, compared, or passed into other functions.

ThinkCScpp 5.2: Program development

- Incremental development builds and tests a program in small working steps.
- Each step should add one limited piece of behaviour.
- Frequent testing makes errors easier to locate.
- Temporary output can verify intermediate values.

ThinkCScpp 5.3: Composition

- Composition uses one expression or function result as part of another operation.
- Small functions can be combined to solve larger problems.

ThinkCScpp 5.4: Overloading

- Function overloading allows several functions to share a name when their parameter lists differ.
- The compiler selects the best matching overload from the supplied arguments.
- Return type alone can't distinguish overloads.
- Ambiguous calls produce compile-time errors.

ThinkCScpp 5.5: Boolean values

- The `bool` type stores either `true` or `false`.
- Comparisons produce Boolean results.
- Boolean values directly represent conditions instead of using numeric substitutes.

ThinkCScpp 5.6: Boolean variables

- Boolean variables store the result of a condition for later use.
- Descriptive names such as `isValid` make decision logic easier to read.
- They can simplify repeated or complex expressions.

ThinkCScpp 5.7: Logical operators

- `&&` requires both operands to be true, while `||` requires at least one.
- `!` reverses a Boolean value.
- Logical operators use short-circuit evaluation.
- Parentheses can clarify how several conditions are grouped.
- Operator precedence can affect results when logical expressions are combined.

ThinkCScpp 5.8: Bool functions

- A Boolean function returns `true` or `false` to describe a condition.
- Predicate names usually read like questions, such as `isEven` or `hasAccess`.
- Returning a comparison directly is often clearer than using an unnecessary `if` statement.
- Boolean functions make complex conditions reusable.

ThinkCScpp 5.9: Returning from main

- Returning `0` from `main` conventionally indicates successful execution.
- A nonzero return value can indicate that the program ended because of an error.

ThinkCScpp 5.10: More recursion

- Recursive functions can return values that depend on smaller recursive results.
- Each call pauses until the next call returns.
- Correct recursion requires both a valid base case and measurable progress.
- Repeated work can make some recursive solutions inefficient.

ThinkCScpp 5.11: Leap of faith

- The recursive leap of faith assumes the smaller recursive call already works correctly.
- The current call then only needs to combine that smaller result into the full solution.
- This approach avoids tracing every recursive level while designing the function.

ThinkCScpp 5.12: One more example

- A complete recursive example combines a base case, a smaller subproblem, and a returned result.
- Tracing several inputs helps confirm that each call moves toward termination.
- Comparing recursive and iterative versions can reveal differences in clarity and cost.
- Edge cases should be tested separately from typical inputs.

2.3.3 Unit 6**ThinkCScpp 6.1: Multiple assignment**

- Reassigning variables updates their stored values during execution.
- The right-hand expression is evaluated before the destination changes.
- Temporary variables may be needed when swapping values.
- Sequential assignments aren't simultaneous.

ThinkCScpp 6.2: Iteration

- Iteration repeatedly executes statements while a condition remains true.
- Loops replace duplicated code when the same operation must occur many times.
- A loop usually needs initialization, a condition, and an update.

ThinkCScpp 6.3: The while statement

- A `while` loop checks its condition before each iteration.
- Its body might not execute at all when the condition starts false.
- The loop body must eventually change something that affects the condition.
- Missing progress can create an infinite loop.
- `while` is useful when the number of iterations isn't known in advance.

ThinkCScpp 6.4: Tables

- A table can be generated by repeating a calculation for a sequence of input values.
- Loops make the range and spacing of table entries easy to control.
- Formatted output keeps columns aligned and readable.

ThinkCScpp 6.5: Two-dimensional tables

- Nested loops generate rows and columns in a two-dimensional table.
- The outer loop usually controls rows while the inner loop controls columns.
- The total work is approximately the product of the two loop counts.
- Each table cell can be calculated from its row and column values.

ThinkCScpp 6.6: Encapsulation and generalization

- Encapsulation moves a repeated block of logic into a named function.
- Generalization replaces fixed values with parameters.
- Together they turn one specific solution into reusable code.
- A generalized function should still have one clear responsibility.

ThinkCScpp 6.7: Functions

- Functions divide a program into named operations with defined inputs and outputs.
- Parameters allow callers to supply data.
- Return values let functions produce reusable results.

ThinkCScpp 6.8: More encapsulation

- Additional encapsulation can separate loop control from the work performed in each iteration.
- Smaller functions reduce duplication and make testing more focused.
- Extracting code is useful when a block has a meaningful independent purpose.
- Excessive fragmentation can make simple logic harder to follow.

ThinkCScpp 6.9: Local variables

- Local variables exist only within the block or function where they're declared.
- Each function call receives its own local variables.
- Limiting scope prevents unrelated code from modifying temporary state.
- A local variable can hide another variable with the same name in an outer scope.
- Variables should be declared near their first use.

ThinkCScpp 6.10: More generalization

- Further generalization identifies additional values or behaviours that can become parameters.
- A generalized function can handle a wider range of inputs without duplicating code.
- Generalization should preserve readability rather than create unnecessary abstraction.

2.4 Practice problems

A full comprehensive list of practice problems can be found in :doc: Practice Problems <PracticeProblems.rst>

2.4.1 Unit 4 practice problems

Fizz Buzz (Easy)

Return each number as text, replacing multiples of three and five with the required words.

```
class Solution {
public:
    vector<string> fizzBuzz(int n) {
        vector<string> answer;

        for (int i = 1; i <= n; ++i) {
            if (i % 15 == 0) {
                answer.push_back("FizzBuzz");
            } else if (i % 3 == 0) {
                answer.push_back("Fizz");
            } else if (i % 5 == 0) {
                answer.push_back("Buzz");
            } else {
                answer.push_back(to_string(i));
            }
        }

        return answer;
    }
};
```

2.4.2 Unit 5 practice problems

Pow(x, n) (Medium)

Calculate a floating-point base raised to an integer exponent using recursion.

```
class Solution {
public:
    double myPow(double x, int n) {
        return power(x, static_cast<long long>(n));
    }

private:
    double power(double x, long long n) {
        if (n == 0) {
            return 1.0;
        }

        if (n < 0) {
            return 1.0 / power(x, -n);
        }

        double half = power(x, n / 2);
```

(continues on next page)

(continued from previous page)

```
    if (n % 2 == 0) {  
        return half * half;  
    }  
  
    return half * half * x;  
}  
};
```

2.4.3 Unit 6 practice problems

Pascal's Triangle (Easy)

Generate the requested number of rows from Pascal's triangle using nested loops.

```
class Solution {  
public:  
    vector<vector<int>> generate(int numRows) {  
        vector<vector<int>> triangle;  
  
        for (int row = 0; row < numRows; ++row) {  
            triangle.push_back(vector<int>(row + 1, 1));  
  
            for (int column = 1; column < row; ++column) {  
                triangle[row][column] =  
                    triangle[row - 1][column - 1] +  
                    triangle[row - 1][column];  
            }  
        }  
  
        return triangle;  
    }  
};
```

2.5 Sources and references

1. cppreference.com. (n.d.). C++ reference. Retrieved Jan 2026 - Aug 2026, from <https://cppreference.com/cpp>
2. Braunschweig, D., & Busbee, K. L. (2018). Programming fundamentals: A modular structured approach (2nd ed.). Rebus Community. <https://press.rebus.community/programmingfundamentals/>
3. Hansen, J. A. (2013). The Rook's guide to C++. Rook's Guide Press. <https://rooksguide.org/>
4. Downey, A. B. (1999). Think C++ (Version 1.1.0). Green Tea Press. <https://www.greenteapress.com/thinkcpp/>
5. leetcode.com. (n.d.). *LeetCode*. Retrieved Jan 2025 - Aug 2026, from <https://leetcode.com/>

ASSIGNMENT THREE JOURNAL

Time spent	Jan 2026 to Aug 2026
Student name	Hector Barquero

Tip

How to use this journal

Each assignment has its own chapter. Use the right chapter to grade the assignment, which has program design notes, changelogs for versions as I made updates, reflections, and reading notes for the units of each assignment. There's also practice questions from the textbook which were recommended to be added to the learning journal for each assignment chapter. If a chapter is missing, it means that assignment isn't done yet because they're being completed sequentially— not all at the same time.

3.1 Program design

3.1.1 Assignment 3, Problem 1

- Added command-line filename handling and documented that choice. (v1.0)
- Opened the requested file with `ifstream` and added an open-failure check. (v1.0-v1.1)
- Counted words with `while (inputFile >> word)` so all whitespace types act as delimiters. (v1.1)
- Added usage output for missing or extra command-line arguments. (v1.1)
- Tested the program with `excerpt.txt`, an empty file and mixed spaces, tabs and line breaks. (v1.2)
- Verified that the supplied `excerpt.txt` contains 327 whitespace-separated words. (v1.2)

3.1.2 Assignment 3, Problem 2

- Added a filename prompt using `getline` so filenames may contain spaces. (v1.0)
- Read the file one complete line at a time with `getline(inputFile, line)`. (v1.0)
- Added a pause after every displayed line that waits for the Enter key. (v1.0-v1.1)
- Added one-based line numbers to make each displayed line easier to verify. (v1.1)
- Added checks for a blank filename and a file that cannot be opened. (v1.1)
- Tested normal text lines, blank lines and the supplied `excerpt.txt`. (v1.2)

3.1.3 Assignment 3, Problem 3

- Added a `TextFileReader` class containing an array of exactly 100 strings. (v1.0)
- Added a default constructor and a filename constructor that reads at most 100 lines. (v1.0)
- Added `contents()` to combine stored lines into one newline-separated string. (v1.1)
- Added `display()` to print each stored line using one-based line numbers. (v1.1)
- Added `isOpen()` and `size()` to support file-error checks and test verification. (v1.1-v1.2)
- Added `TextFileReaderDemo::run()` and forwarded the required global C++ `main()` to it. (v1.2)
- Tested the default constructor, missing files, short files and the 100-line storage limit. (v1.3)

3.1.4 Assignment 3, Problem 4

- Added three `array<double, 25>` objects for counter values, squares and sums. (v1.0)
- Filled the first array with counter values from 0 through 24. (v1.0)
- Filled the second array using `value * value` for each corresponding element. (v1.0)
- Filled the third array by adding corresponding elements from the first two arrays. (v1.0)
- Added formatted output using `counter; element + element = element`. (v1.1)
- Verified the first, middle and final rows and confirmed that exactly 25 rows are displayed. (v1.1)

3.1.5 Assignment 3, Problem 5

- Reused the private `Book` attributes and public `get/set` methods from Assignment 2. (v1.0)
- Added a `Bookshelf` class containing a vector of twelve distinct `Book` objects. (v1.0)
- Displayed all twelve books in their original insertion order. (v1.1)
- Added `BookComparator::compareTo()` to compare titles first and publication years second. (v1.1-v1.2)
- Connected the comparator to `std::sort` through `operator()`. (v1.2)
- Added two books with the same title to verify that the earlier publication year sorts first. (v1.3)
- Verified the unsorted and sorted lists both contain all twelve books and all stored attributes. (v1.4)

3.2 Overview and reflections

3.2.1 Unit 7: Points to ponder

Hector Barquero posted Mar 21, 2026 8:35 PM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/350740/View>

I compared differences to javascript and the main differences I found were mutability. I forgot that there was a subtle difference, you can't mutate strings in js, but you can in cpp.

I also forgot that `s.find` is a member function in cpp and returns `string::npos` when there's nothing found, but in javascript it returns -1 (`indexOf`) and is a method you use on the string itself.

3.2.2 Unit 8: Points to ponder

Hector Barquero posted Jun 8, 2026 3:42 PM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/391920/View>

Reading this chapter took me a long time, since I took a small hiatus from AU coursework– but I’m back and was pleasantly surprised that things were ramping up in this chapter since we’ve moved from thinking about “just writing code” to now thinking about the tech debt of our mathematical calculations.

Instead of passing regular vars, we’re now looking at structs and compound types, doing things like call by values and call by refs.

This is fun for me because compared to javascript, c++ makes these concepts explicit. In js, objects are passed around and object shapes change at runtime whereas c++ forces you to think about the structure of your data, if you want to copy it, and how you’ll pass it within functions. It feels like a step away for me from writing code to now writing calculations, and that’s fun.

3.2.3 Unit 9: Points to ponder

Hector Barquero posted Jun 12, 2026 4:05 AM <https://learning.athabascau.ca/d2l/le/18658/discussions/threads/394111/View>

I found this chapter a lighter read since it was less about syntax and more about design choices. One neat difference from what I’m used to is that c++ lets you declare const with a reference, for example:

```
const &foo n
```

where you’re essentially saying: “hey use this original object, don’t copy it, and make it immutable”. Javascript doesn’t have that, so it’s neat for me to remember that’s a thing. Objects in js are mutable and passed around with reference-like behaviour by default, but in c++ you’re constantly thinking about choices between copying, modifying the original or a read only-like behaviour:

```
for ex: foo n <- copy foo &n <- modify original const n <- read only
```

This chapter was cool because it exposed design patterns and thinking less than syntax, and exposed what other languages like javascript are hiding from you.

3.3 Readings

3.3.1 Unit 7

ThinkCScpp 7.1: Containers for strings

- A string stores an ordered sequence of characters in one object.
- `std::string` manages its own storage and can grow or shrink as needed.

ThinkCScpp 7.2: string variables

- A string variable can be declared, initialized, reassigned, and passed to functions.
- String literals use double quotes.
- `std::string` belongs to the standard library.

ThinkCScpp 7.3: Extracting characters from a string

- The subscript operator `[]` accesses a character by its zero-based index.
- The first character is at index `0`.
- Accessing an invalid index produces undefined behaviour.

ThinkCScpp 7.4: Length

- The `length` or `size` function returns the number of characters in a string.
- The final valid index is one less than the string's length.

ThinkCScpp 7.5: Traversal

- String traversal visits characters one at a time, usually with a loop.
- An index-based loop can read or modify each character.
- A range-based loop is useful when the index isn't needed.
- Traversal normally runs in linear time relative to the string length.

ThinkCScpp 7.6: A run-time error

- A run-time error can occur when a program accesses a string outside its valid range.
- The code may compile even though the invalid index only appears during execution.
- Boundary tests help expose off-by-one errors.

ThinkCScpp 7.7: The find function

- `string::find` returns the first position where a character or substring appears.
- It returns `string::npos` when no match exists.
- Searches can optionally begin at a specified index.
- The result should be checked before it's used as an index.

ThinkCScpp 7.8: Our own version of find

- A custom find function scans characters until it locates the target.
- It should return either the matching position or a clear failure value.
- Writing the search manually demonstrates traversal, conditions, and early return.

ThinkCScpp 7.9: Looping and counting

- A counter can track how many characters satisfy a condition during traversal.
- The counter starts at zero and increases when a match is found.
- Counting requires examining every character unless an earlier result is sufficient.
- The operation is generally $O(n)$.

ThinkCScpp 7.10: Increment and decrement operators

- `++` adds one to a variable and `--` subtracts one.
- Prefix and postfix forms differ in the value produced by the expression.
- In simple loop updates, either form usually has the same practical effect.

ThinkCScpp 7.11: String concatenation

- The `+` operator joins strings into a new string.
- `+=` appends text to an existing string.
- At least one operand must be a `string` for many concatenation expressions.
- Repeated concatenation can require additional memory allocations.

ThinkCScpp 7.12: strings are mutable

- Individual string characters can be changed through valid indexes.
- Strings can also be expanded, shortened, inserted into, or erased.
- Mutation changes the existing string rather than creating a separate result.

ThinkCScpp 7.13: strings are comparable

- Comparison operators compare strings lexicographically.
- Character case affects comparison results.
- Equality requires the same characters in the same order.
- Lexicographic order isn't the same as comparing string lengths.

ThinkCScpp 7.14: Character classification

- Functions such as `isalpha`, `isdigit`, and `isspace` classify characters.
- `toupper` and `tolower` convert character case.
- These functions are provided by `<cctype>`.
- Character arguments should be converted to `unsigned char` when plain `char` might be negative.
- Classification functions return values usable as Boolean conditions.

ThinkCScpp 7.15: Other string functions

- Common string operations include `substr`, `insert`, `erase`, `replace`, and `append`.
- `substr` extracts part of a string using a starting position and optional length.
- Many functions return or modify positions measured from zero.

3.3.2 Unit 8**ThinkCScpp 8.1: Compound values**

- A compound value groups several related values into one object.
- Structures can combine members with different data types.
- Grouping related data makes function interfaces and program models clearer.

ThinkCScpp 8.2: Point objects

- A `Point` structure can represent coordinates using members such as `x` and `y`.
- Each object stores its own independent member values.

ThinkCScpp 8.3: Accessing instance variables

- The dot operator accesses a structure object's members.
- Member names are scoped by the structure type.
- A pointer to a structure uses the `->` operator instead.

ThinkCScpp 8.4: Operations on structures

- Structures can be copied, assigned, initialized, and passed to functions.
- Member values can participate in ordinary expressions.
- Comparison isn't automatically defined for every custom structure.
- Helper functions can implement operations specific to the structure.

ThinkCScpp 8.5: Structures as parameters

- Passing a structure to a function lets the function operate on the grouped data.
- The parameter type must match the structure type.
- Large structures are often passed by reference to avoid copying.

ThinkCScpp 8.6: Call by value

- Call by value gives the function a copy of the argument.
- Changes to the parameter don't affect the original object.
- Copying provides isolation but can add cost for large objects.

ThinkCScpp 8.7: Call by reference

- A reference parameter gives the function access to the caller's original object.
- Changes made through a non-const reference affect the caller.
- References avoid copying and can't be null after valid initialization.
- Use const references when the function should only read the object.

ThinkCScpp 8.8: Rectangles

- A rectangle can be represented using a corner point, width, and height.
- Nested structures let one compound object contain another.
- Representation choices affect how position and size calculations are written.

ThinkCScpp 8.9: Structures as return types

- A function can construct and return a complete structure object.
- Returning a structure is useful when one calculation produces several related values.
- Modern compilers usually optimize away unnecessary return copies.

ThinkCScpp 8.10: Passing other types by reference

- Reference parameters aren't limited to structures and can be used with built-in or library types.
- They can return multiple results through output parameters.
- `const` references allow efficient read-only access.
- Non-`const` references should clearly signal that an argument may change.

ThinkCScpp 8.11: Getting user input

- Input can be read directly into structure members.
- Each extraction should be checked before the values are trusted.
- Prompts should make the expected format clear.
- Invalid input can leave the stream in a failed state.
- `getline` may be needed when text fields contain spaces.

3.3.3 Unit 9**ThinkCScpp 9.1: Time**

- A `Time` structure can group hours, minutes, and seconds.
- The representation should define valid ranges for each member.
- Related values are easier to pass and return as one object.

ThinkCScpp 9.2: `printTime`

- A `printTime` function formats a `Time` object for output.
- Separating formatting from calculation keeps responsibilities clear.
- Padding may be used so minutes and seconds always occupy two digits.

ThinkCScpp 9.3: Functions for objects

- Functions can accept objects as parameters and perform type-specific operations.
- Related helper functions form an interface around the object's representation.
- The function name should describe the operation rather than the storage details.

ThinkCScpp 9.4: Pure functions

- A pure function returns a result without modifying its arguments or external state.
- The same inputs always produce the same output.
- Pure functions are easier to test and reuse.
- They reduce unexpected interactions between parts of a program.

ThinkCScpp 9.5: `const` parameters

- A `const` reference parameter prevents the function from modifying the supplied object.
- It avoids copying while documenting read-only intent.
- The compiler rejects attempts to change the parameter.

ThinkCScpp 9.6: Modifiers

- A modifier changes an existing object rather than returning a separate object.
- Modifier functions usually take non-const references.
- Their names should make the mutation clear.
- Callers must account for the original object's changed state.

ThinkCScpp 9.7: Fill-in functions

- A fill-in function writes a result into an object supplied by reference.
- It can avoid constructing a separate return value.
- Output parameters should be obvious from the function interface.

ThinkCScpp 9.8: Which is best?

- Pure functions are usually easier to reason about because they don't change their inputs.
- Modifiers can be useful when updating an existing object is the intended operation.
- Fill-in functions may help when several results must be returned.
- The best design depends on clarity, ownership, cost, and expected use.
- Consistency matters more than forcing one style everywhere.

ThinkCScpp 9.9: Incremental development versus planning

- Incremental development adds and tests one small behaviour at a time.
- Up-front planning defines the representation and major operations before implementation.
- Complex tasks often benefit from combining both approaches.
- Tests should be added as each step becomes functional.

ThinkCScpp 9.10: Generalization

- Generalization replaces fixed assumptions with parameters or reusable operations.
- A generalized function can handle more cases without duplicated code.
- Abstraction shouldn't make a simple solution harder to understand.

ThinkCScpp 9.11: Algorithms

- An algorithm is a finite, defined sequence of steps for solving a problem.
- Algorithms can be compared by correctness, clarity, time cost, and memory cost.
- The same problem may support several valid algorithms.
- Input size determines how performance scales.

3.4 Practice problems

3.4.1 Unit 7 practice problems

Find the Index of the First Occurrence in a String (Easy)

Return the first index where one string occurs inside another, or `-1` when it isn't found.


```

class Solution {
public:
    int strStr(string haystack, string needle) {
        if (needle.empty()) {
            return 0;
        }

        if (needle.size() > haystack.size()) {
            return -1;
        }

        for (size_t start = 0;
             start + needle.size() <= haystack.size();
             ++start) {

            bool matches = true;

            for (size_t offset = 0;
                 offset < needle.size();
                 ++offset) {

                if (haystack[start + offset] != needle[offset]) {
                    matches = false;
                    break;
                }
            }

            if (matches) {
                return static_cast<int>(start);
            }
        }

        return -1;
    }
};

```

3.4.2 Unit 8 practice problems

Rectangle Overlap (Easy)

Determine whether two rectangles share an area greater than zero.

```

class Solution {
private:
    struct Point {
        int x;
        int y;
    };

    struct Rectangle {
        Point bottomLeft;
        Point topRight;
    };

```

(continues on next page)

(continued from previous page)

```

Rectangle makeRectangle(const vector<int>& values) {
    return {
        {values[0], values[1]},
        {values[2], values[3]}
    };
}

bool overlaps(const Rectangle& first,
              const Rectangle& second) {
    bool separatedHorizontally =
        first.topRight.x <= second.bottomLeft.x ||
        second.topRight.x <= first.bottomLeft.x;

    bool separatedVertically =
        first.topRight.y <= second.bottomLeft.y ||
        second.topRight.y <= first.bottomLeft.y;

    return !separatedHorizontally &&
           !separatedVertically;
}

public:
    bool isRectangleOverlap(vector<int>& rec1,
                           vector<int>& rec2) {
        Rectangle first = makeRectangle(rec1);
        Rectangle second = makeRectangle(rec2);

        return overlaps(first, second);
    }
};

```

3.4.3 Unit 9 practice problems

Number of Days Between Two Dates (Easy)

Calculate the absolute number of days between two dates written as ``YYYY-MM-DD``.

```

class Solution {
private:
    struct Date {
        int year;
        int month;
        int day;
    };

    Date parseDate(const string& value) {
        return {
            stoi(value.substr(0, 4)),
            stoi(value.substr(5, 2)),
            stoi(value.substr(8, 2))
        };
    }
};

```

(continues on next page)

(continued from previous page)

```
}

bool isLeapYear(int year) {
    return year % 400 == 0 ||
           (year % 4 == 0 && year % 100 != 0);
}

int daysBeforeYear(int year) {
    int previousYear = year - 1;

    return previousYear * 365 +
           previousYear / 4 -
           previousYear / 100 +
           previousYear / 400;
}

int daysBeforeMonth(const Date& date) {
    const array<int, 12> monthLengths = {
        31, 28, 31, 30, 31, 30,
        31, 31, 30, 31, 30, 31
    };

    int total = 0;

    for (int month = 1; month < date.month; ++month) {
        total += monthLengths[month - 1];

        if (month == 2 && isLeapYear(date.year)) {
            ++total;
        }
    }

    return total;
}

int serialDay(const Date& date) {
    return daysBeforeYear(date.year) +
           daysBeforeMonth(date) +
           date.day;
}

public:
    int daysBetweenDates(string date1, string date2) {
        Date first = parseDate(date1);
        Date second = parseDate(date2);

        return abs(serialDay(first) - serialDay(second));
    }
};
```

3.5 Sources and references

1. cppreference.com. (n.d.). C++ reference. Retrieved Jan 2026 - Aug 2026, from <https://cppreference.com/cpp>
2. Braunschweig, D., & Busbee, K. L. (2018). Programming fundamentals: A modular structured approach (2nd ed.). Rebus Community. <https://press.rebus.community/programmingfundamentals/>
3. Hansen, J. A. (2013). The Rook's guide to C++. Rook's Guide Press. <https://rooksguide.org/>
4. Downey, A. B. (1999). Think C++ (Version 1.1.0). Green Tea Press. <https://www.greenteapress.com/thinkcpp/>
5. leetcode.com. (n.d.). *LeetCode*. Retrieved Jan 2025 - Aug 2026, from <https://leetcode.com/>

ASSIGNMENT FOUR JOURNAL

Warning

This assignment isn't submitted yet, don't grade this part and you can ignore this chapter.

The chapter is empty for now. On the submission for Assignment 4, this gets updated with the .zip contents of assignment 4.

I'm using reStructuredText and a sphinx documentation project to write docs as I go, so that's why this is staged and empty.

Each assignment has it's own chapter. Use the right chapter to grade the assignment, which has program design notes, changelogs for versions as I made updates, reflections, and reading notes for the units of each assignment. There's also practice questions from the textbook which were recommended to be added to the learning journal for each assignment chapter. If a chapter is missing, it means that assignment isn't done yet because they're being completed sequentially– not all at the same time.